

Planificación y modelado conceptual y lógico de estructuras de datos

Breve descripción:

Este componente capacita al aprendiz en la ingeniería de datos inicial, abarcando desde la gestión del proyecto hasta la transformación relacional. Mediante la estructuración de planes de trabajo y el dominio del Modelo Entidad-Relación y esquemas lógicos, el estudiante resolverá problemas del entorno productivo, garantizando que las estructuras tecnológicas diseñadas respondan fielmente a los requerimientos y reglas del negocio corporativo.

Tabla de contenido

Introducción	5
1. Introducción a los sistemas de almacenamiento y ciclo de vida	7
1.1. Evolución histórica del almacenamiento de datos y limitaciones del enfoque tradicional	7
1.2. Archivos planos en la actualidad	10
1.3. Emergencia del paradigma de bases de datos y conceptualización de los SGBD.....	11
1.4. Arquitectura de tres niveles ANSI-SPARC y terminología fundamental	12
1.5. Propiedades ACID y robustez transaccional en entornos corporativos	15
2. Planificación técnica y gestión de proyectos de diseño de estructuras de datos	20
2.1. Metodologías de planificación y el ciclo de vida del desarrollo de software (CVDS)	22
2.2. Elaboración técnica del plan de trabajo: actividades, secuenciación y dependencias	24
2.3. Herramientas de gestión y diagramas de planificación: Gantt y estimación de recursos	26
3. Modelado conceptual de datos: componentes esenciales del Modelo Entidad-Relación (MER)	31
3.1. Teoría de conjuntos y conceptualización de entidades fuertes y débiles ..	31

3.2. Taxonomía exhaustiva de atributos: simples, compuestos, multivaluados y derivados	34
3.3. Teoría formal de claves para la integridad semántica de los datos	36
3.4. Restricción de dominio de un atributo	38
4. Semántica avanzada y extensiones en el modelado conceptual (MERE)	41
4.1. El concepto de relación: grado, cardinalidad y restricciones de participación	41
4.2. Extensiones del Modelo (MERE): jerarquías de generalización y especialización.....	47
4.3. Semántica de la agregación y restricciones de exclusividad en relaciones	49
5. Ingeniería de software relacional: metodología de transformación lógica y herramientas CASE	54
5.1. Arquitectura del modelo lógico relacional y la teoría de Codd.....	54
5.2. Reglas de transformación formal del MER/MERE al modelo relacional de tablas	57
5.3. Criterios de selección e Ingeniería de Software Asistida por Computadora (Herramientas CASE)	58
Síntesis	61
Glosario	62
Referencias bibliográficas	64



Créditos	66
----------------	----

Introducción

En la actualidad, las bases de datos constituyen uno de los pilares fundamentales de los sistemas de información utilizados por organizaciones de todos los sectores productivos. Desde aplicaciones financieras y plataformas de comercio electrónico hasta sistemas de salud, educación y logística, la gestión eficiente de los datos permite apoyar la toma de decisiones, optimizar procesos y garantizar la disponibilidad de información confiable. Por esta razón, el diseño de estructuras de datos requiere una metodología que combine el análisis de las necesidades del negocio con principios técnicos orientados a la calidad, la integridad y la seguridad de la información.

El desarrollo de una base de datos relacional comprende un proceso organizado que inicia con la identificación de los requerimientos de información y continúa con la planificación del proyecto, el modelado conceptual y la transformación de los modelos en estructuras lógicas listas para su implementación en un Sistema de Gestión de Bases de Datos (SGBD). Durante este proceso se aplican metodologías, reglas de modelado y herramientas especializadas que permiten representar correctamente las entidades, los atributos, las relaciones y las restricciones definidas por las reglas de negocio de una organización.

Este componente formativo proporciona los fundamentos necesarios para comprender el ciclo de vida del diseño de bases de datos relacionales, desde la evolución de los sistemas de almacenamiento hasta la aplicación del Modelo Entidad-Relación (MER), el Modelo Entidad-Relación Extendido (MERE) y las reglas de transformación hacia el modelo relacional. Asimismo, aborda la planificación técnica de proyectos y el uso de herramientas CASE como apoyo al proceso de diseño, con el

propósito de desarrollar soluciones de almacenamiento de datos consistentes, escalables y alineadas con las necesidades del sector productivo.

1. Introducción a los sistemas de almacenamiento y ciclo de vida

En el ecosistema tecnológico contemporáneo, la información se ha consolidado como el activo estratégico más valioso de las organizaciones, al constituirse en el insumo crítico para la automatización de procesos corporativos, el desarrollo de aplicaciones de software y la analítica predictiva de negocios.

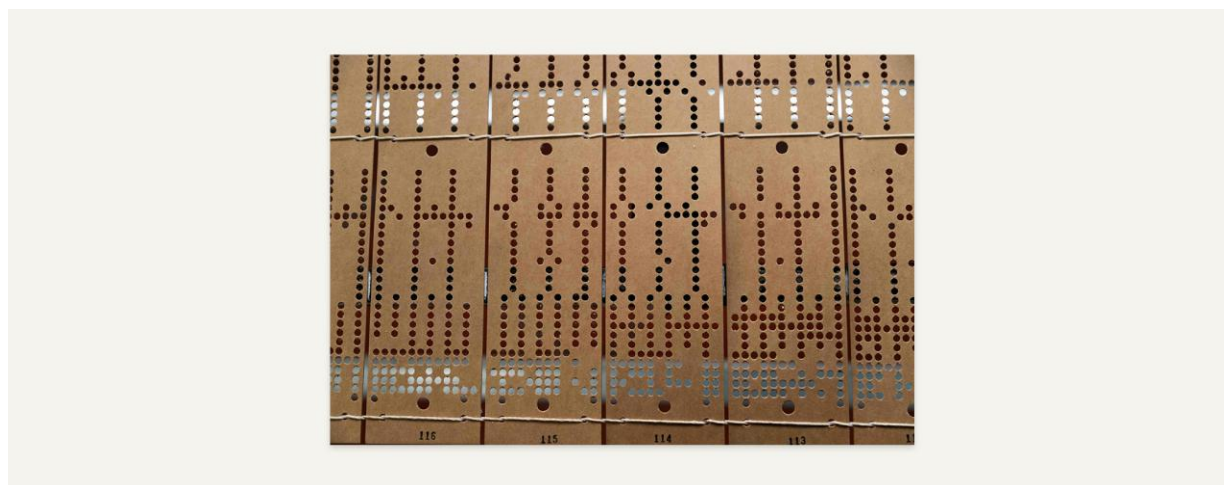
Para comprender con rigor metodológico el valor de las arquitecturas de datos modernas, resulta necesario analizar la evolución de las tecnologías de almacenamiento y las limitaciones que impulsaron la transición hacia los paradigmas actuales. Este recorrido permite entender cómo surgieron las bases de datos como respuesta a las necesidades de integridad, disponibilidad y gestión eficiente de la información.

1.1. Evolución histórica del almacenamiento de datos y limitaciones del enfoque tradicional

Durante las décadas de 1950 y 1960, el almacenamiento y procesamiento de la información se realizaba mediante el denominado enfoque tradicional de sistemas de ficheros o archivos planos. Bajo esta arquitectura descentralizada, cada departamento o aplicación administraba y estructuraba de forma independiente sus propios archivos físicos de datos, los cuales, comúnmente, se almacenaban en cintas magnéticas o tarjetas perforadas.

Aunque este modelo permitió las primeras automatizaciones de los procesos informáticos, el crecimiento de las organizaciones evidenció importantes limitaciones estructurales que afectaban la calidad, consistencia y disponibilidad de la información. El siguiente recurso corresponde a uno de los medios de almacenamiento utilizados durante los primeros años de la informática administrativa.

Figura 1. Tarjetas perforadas para guardar datos binarios



Entre las principales limitaciones del enfoque tradicional se encuentran las siguientes:

- **Redundancia e inconsistencia de datos.** La misma información se almacenaba en múltiples archivos independientes. Por ejemplo, los datos básicos de un cliente, como el nombre, la dirección corporativa y el número de contacto, podían registrarse tanto en el sistema de facturación como en el sistema de despachos. Esta duplicidad favorecía la aparición de inconsistencias cuando una actualización se realizaba en un archivo, pero no en los demás. Como consecuencia, el sistema podía contener información contradictoria, lo que afectaba la integridad de los datos y la confiabilidad de los procesos organizacionales.
- **Dependencia físico-lógica entre datos y aplicaciones.** La estructura lógica y la organización física de los registros se definían directamente en el código fuente de cada aplicación. Aspectos como el tamaño de los campos, el tipo de codificación de caracteres o la disposición de los registros en el disco

estaban estrechamente ligados al software. En consecuencia, cualquier modificación en la estructura de un archivo físico, por ejemplo, la ampliación de un campo de identificación de ocho a diez dígitos para cumplir una nueva disposición legal, obligaba a modificar, recompilar y redistribuir todas las aplicaciones que interactuaban con dicho archivo. Esta dependencia incrementaba los costos de mantenimiento del software y reducía la capacidad de adaptación del sistema informático frente a los cambios del entorno empresarial.

- **Limitaciones operativas.** Los archivos planos carecían de mecanismos centralizados para administrar el acceso concurrente, controlar la seguridad y recuperar la información ante fallos. Si dos aplicaciones modificaban simultáneamente un mismo registro, podían producirse conflictos de sobreescritura que comprometían la integridad de los datos. Además, la ausencia de controles centralizados dificultaba la implementación de políticas de seguridad para restringir el acceso a usuarios específicos. Finalmente, una interrupción ocasionada por una falla eléctrica o un error de hardware podía dejar los archivos en un estado inconsistente o parcialmente actualizado, sin mecanismos automáticos para recuperar la información.

Como resultado de estas limitaciones, el enfoque basado en archivos planos dejó de responder a las crecientes necesidades de las organizaciones. La redundancia de la información, la dependencia entre datos y aplicaciones, así como las dificultades para garantizar la integridad, la seguridad y la recuperación de los datos, impulsaron el

desarrollo del paradigma de las bases de datos, tema que se abordará en el siguiente apartado.

1.2. Archivos planos en la actualidad

Aunque los archivos planos surgieron en las primeras etapas de la informática, continúan siendo una alternativa útil para el almacenamiento y el intercambio de información en distintos entornos tecnológicos. Su simplicidad, compatibilidad y facilidad de procesamiento hacen que todavía sean ampliamente utilizados en procesos de integración y administración de datos.

Un archivo plano de datos, como un archivo .txt o .csv, permite almacenar y transferir información de forma sencilla, sin formatos ni estructuras complejas. A pesar de tratarse de una tecnología con varias décadas de existencia, sigue siendo un recurso fundamental en la informática moderna debido a su compatibilidad universal y ligereza (Sipecom, 2026). Entre sus principales usos en la actualidad se encuentran los siguientes:

- **Migración de datos.** Facilita la transferencia de información entre sistemas heredados y plataformas más recientes.
- **Cargas masivas de información.** Permite importar grandes volúmenes de productos, clientes o usuarios a aplicaciones y plataformas web.
- **Integración de sistemas.** Favorece el intercambio de información entre aplicaciones desarrolladas por diferentes fabricantes o tecnologías.
- **Copias de seguridad.** Almacena respaldos históricos de información en archivos de bajo tamaño y fácil conservación.

- **Análisis de datos.** Facilita la importación de grandes volúmenes de datos sin procesar en herramientas de análisis, como Python, R o Excel.

Como puede apreciarse, los archivos planos continúan desempeñando un papel importante en la informática actual, especialmente en procesos de migración, integración de sistemas, respaldo de información y análisis de datos, donde su simplicidad y compatibilidad representan ventajas significativas.

1.3. Emergencia del paradigma de bases de datos y conceptualización de los SGBD

Con el propósito de superar las limitaciones del sistema tradicional de archivos, la ingeniería de software dio paso al paradigma de las bases de datos (BD). Una base de datos se define como una colección integrada, estructurada y compartida de datos lógicamente relacionados, acompañada de una descripción de dichos datos, denominada metadatos, diseñada para satisfacer las necesidades de información de una organización.

A diferencia de los ficheros dispersos, una base de datos centraliza la información en un repositorio lógico común, al que pueden acceder simultáneamente múltiples aplicaciones y diferentes perfiles de usuarios. Esta organización mejora la consistencia de los datos, facilita su administración y reduce la duplicidad de la información.

La administración, la seguridad y el acceso eficiente a este repositorio no dependen de las aplicaciones individuales, sino de un Sistema de Gestión de Bases de Datos (SGBD) o Database Management System (DBMS). El SGBD actúa como una capa intermedia entre la base de datos almacenada físicamente, los usuarios y las aplicaciones que utilizan la información. Su función consiste en proporcionar un

entorno seguro y eficiente para almacenar, recuperar y modificar los datos. Para cumplir este propósito, un SGBD incorpora diferentes lenguajes especializados que permiten definir y manipular la información de la base de datos:

- **Lenguaje de Definición de Datos (DDL, Data Definition Language).**

Permite definir la estructura lógica de la base de datos, crear tablas, establecer restricciones de integridad y configurar mecanismos de seguridad y acceso.

- **Lenguaje de Manipulación de Datos (DML, Data Manipulation Language).** Permite consultar, insertar, actualizar y eliminar los datos almacenados en la base de datos.

Una de las principales ventajas de implementar un SGBD es la independencia de los datos. Esto significa que la representación lógica de la información permanece separada de la forma en que los datos se almacenan físicamente en los dispositivos de almacenamiento.

Gracias a esta independencia, el administrador de la base de datos puede reorganizar los archivos físicos, modificar los métodos de indexación o migrar el hardware del servidor para optimizar el rendimiento del sistema, sin necesidad de reescribir o recompilar las aplicaciones que utilizan la información almacenada en el SGBD.

1.4. Arquitectura de tres niveles ANSI-SPARC y terminología fundamental

Con el propósito de garantizar la independencia de los datos y permitir que diferentes usuarios accedan a la información según sus necesidades, el comité Standards Planning and Requirements Committee (ANSI-SPARC) propuso una

arquitectura de referencia que organiza el diseño de un Sistema de Gestión de Bases de Datos (SGBD) en tres niveles lógicos de abstracción. Esta arquitectura facilita la administración de la información, mejora la seguridad y permite que cada usuario interactúe únicamente con los datos que requiere. Los tres niveles de la arquitectura ANSI-SPARC se describen a continuación:

- **Nivel externo o de vistas.** Es la capa más cercana a los usuarios finales. Define diferentes vistas de la base de datos según el perfil de cada usuario. Por ejemplo, en un sistema de información educativa, el aprendiz puede consultar únicamente sus calificaciones e historial académico, mientras que el instructor dispone de permisos para registrar y modificar dicha información. De esta manera, cada usuario accede exclusivamente a los datos que le corresponden.
- **Nivel conceptual.** Representa la estructura lógica global de la base de datos, sin considerar los detalles de almacenamiento físico. En este nivel se define qué información se almacena, cómo se relacionan los datos y cuáles son las restricciones de integridad que debe cumplir el sistema. Constituye el núcleo del proceso de modelado relacional.
- **Nivel interno o físico.** Corresponde al nivel de menor abstracción y describe la forma en que los datos se almacenan físicamente en los dispositivos de hardware. Incluye aspectos como la asignación de espacio en memoria, la organización de los archivos, los algoritmos de compresión y la configuración de los índices de búsqueda, utilizados para optimizar el acceso a la información.

Para comprender el funcionamiento de esta arquitectura, es necesario familiarizarse con algunos conceptos fundamentales utilizados en el diseño y administración de bases de datos:

- **Esquema de base de datos.** Es la descripción formal de la estructura lógica de la base de datos. Se diseña durante las primeras etapas del proyecto y permanece relativamente estable durante el ciclo de vida del sistema. Define las tablas, los atributos, las relaciones y las restricciones de integridad que conforman la estructura del repositorio de datos.
- **Instancia de base de datos.** Corresponde al conjunto de datos almacenados en un momento determinado. A diferencia del esquema, la instancia cambia continuamente como resultado de las operaciones de inserción, actualización y eliminación realizadas por los usuarios o las aplicaciones.
- **Metadatos.** Son los datos que describen otros datos. Se almacenan en el catálogo o diccionario de datos del SGBD e incluyen información sobre las tablas, los tipos de datos, la longitud de los campos, las restricciones de integridad y otros elementos necesarios para administrar la base de datos.

El modelo ANSI-SPARC continúa siendo un referente para comprender la separación entre la estructura lógica y el almacenamiento físico de la información. Esta organización favorece la independencia de los datos, simplifica el mantenimiento de las aplicaciones y facilita la administración de los sistemas de bases de datos en entornos organizacionales.

1.5. Propiedades ACID y robustez transaccional en entornos corporativos

Para que una base de datos pueda utilizarse de forma confiable en sectores como el financiero, el logístico o el de la salud, el Sistema de Gestión de Bases de Datos (SGBD) debe garantizar la integridad y la disponibilidad de la información, incluso cuando múltiples usuarios ejecutan operaciones simultáneamente o se presentan fallas inesperadas. Este nivel de confiabilidad se fundamenta en el cumplimiento de las propiedades ACID, un conjunto de principios que garantizan la correcta ejecución de las transacciones.

Una transacción es una unidad lógica de trabajo compuesta por una secuencia de operaciones de manipulación de datos, como consultas, inserciones, actualizaciones o eliminaciones. Estas operaciones deben ejecutarse por completo o, en caso de presentarse algún error, anularse totalmente para preservar la integridad de la información. Las propiedades ACID se describen a continuación:

- **Atomicidad.** Garantiza que una transacción se ejecute como una unidad indivisible. El SGBD registra cada operación en un log transaccional y, si ocurre un fallo antes de finalizar el proceso, ejecuta un rollback o reversión transaccional para deshacer todos los cambios realizados. De esta manera, la base de datos retorna al estado en que se encontraba antes del inicio de la transacción. En otras palabras, la transacción se completa en su totalidad o no produce ningún cambio.
- **Consistencia.** Asegura que toda transacción lleve la base de datos desde un estado válido hasta otro estado igualmente válido. Durante este proceso, el SGBD verifica el cumplimiento de las restricciones de integridad, como la unicidad de las llaves primarias, la correspondencia de las llaves foráneas y

las reglas de validación de los datos. Si alguna de estas condiciones no se cumple, la transacción es cancelada y los cambios no se almacenan.

- **Aislamiento.** Garantiza que las transacciones ejecutadas de manera simultánea no interfieran entre sí. Para lograrlo, el SGBD utiliza mecanismos de bloqueo de registros (locking) o técnicas de control de concurrencia. Mientras una transacción modifica un registro, las demás deben esperar hasta que la operación finalice. De esta forma, cada transacción se ejecuta como si fuera la única que utiliza la base de datos en ese momento.
- **Durabilidad.** Establece que, una vez confirmada una transacción mediante la operación commit, los cambios realizados se vuelven permanentes. Incluso si ocurre una falla del hardware, una interrupción del suministro eléctrico o un error del sistema operativo, el SGBD recupera la información utilizando el log transaccional, lo que garantiza que los datos confirmados no se pierdan.

El cumplimiento de estas propiedades permite que los sistemas de bases de datos mantengan la integridad, la consistencia y la disponibilidad de la información, incluso en entornos donde se procesan grandes volúmenes de transacciones de manera simultánea. Por esta razón, las propiedades ACID constituyen uno de los fundamentos más importantes para el diseño y la administración de bases de datos en el sector productivo.

Caso de uso en el sector productivo: sistema transaccional de Banco Financiero S. A.

Para comprender la importancia de las propiedades ACID en un entorno real, se presenta el caso de una entidad bancaria que procesa diariamente millones de operaciones de depósitos, retiros y transferencias mediante cajeros automáticos,

aplicaciones móviles y oficinas de atención al cliente. En este contexto, la correcta ejecución de las transacciones es fundamental para garantizar la integridad de la información y la confianza de los usuarios.

Considere la siguiente regla de negocio: un cliente desea transferir \$500.000 desde la Cuenta A hacia la Cuenta B. Desde la perspectiva de un sistema de bases de datos, esta operación no corresponde a una única acción, sino a una transacción conformada por varias operaciones de manipulación de datos (DML), las cuales deben ejecutarse en un orden específico:

- **Operación 1 - Lectura.** Verificar que la Cuenta A disponga de un saldo igual o superior a \$500.000.
- **Operación 2 - Modificación.** Debitar \$500.000 del saldo disponible en la Cuenta A.
- **Operación 3 - Lectura.** Validar que la Cuenta B exista y se encuentre activa en el sistema.
- **Operación 4 - Modificación.** Acreditar \$500.000 al saldo de la Cuenta B.

Si esta operación se realizara mediante un sistema basado en archivos planos y ocurriera una interrupción del suministro eléctrico inmediatamente después de ejecutar la segunda operación, el dinero sería descontado de la Cuenta A, pero nunca llegaría a la Cuenta B. Como consecuencia, la información quedaría inconsistente, se producirían diferencias contables y el banco tendría que enfrentar reclamaciones por parte de sus clientes.

Si durante la ejecución de estas operaciones ocurriera una interrupción del suministro eléctrico inmediatamente después de realizar el débito en la Cuenta A, el

dinero sería descontado, pero no alcanzaría a acreditarse en la Cuenta B. Como consecuencia, la información quedaría inconsistente, se generarían diferencias contables y la entidad tendría que atender reclamaciones por parte de los clientes. Este tipo de situaciones evidencia la importancia de contar con mecanismos que garanticen la confiabilidad de las transacciones:

- **Atomicidad.** Agrupa las cuatro operaciones en una única transacción. Si ocurre una falla antes de finalizar el proceso, el SGBD ejecuta un proceso de rollback, revierte todos los cambios realizados y restablece la base de datos al estado anterior al inicio de la transacción.
- **Consistencia.** Garantiza que la transferencia solo se complete cuando se cumplen todas las reglas de integridad definidas. Por ejemplo, impide ejecutar la operación si la Cuenta A no dispone de saldo suficiente o si la Cuenta B no existe o se encuentra inactiva.
- **Aislamiento.** Evita que otras transacciones consulten o modifiquen la información mientras la transferencia está en ejecución. De esta manera, ningún usuario accede a saldos parcialmente actualizados ni a estados intermedios que puedan generar inconsistencias.
- **Durabilidad.** Garantiza que, una vez confirmada la transferencia mediante la operación commit, los cambios permanezcan almacenados de forma permanente. Incluso si ocurre una falla del hardware o del sistema, el SGBD utiliza sus mecanismos de recuperación para conservar la información previamente confirmada.

Como puede apreciarse, las propiedades ACID permiten que las transacciones se ejecuten de forma segura, íntegra y confiable. Gracias a estos principios, los Sistemas

de Gestión de Bases de Datos pueden procesar grandes volúmenes de operaciones simultáneas, preservar la consistencia de la información y garantizar la continuidad de los procesos críticos en organizaciones del sector productivo.

2. Planificación técnica y gestión de proyectos de diseño de estructuras de datos

A continuación, se presenta un pódcast sobre diseño de estructuras de datos desde la perspectiva de la planificación del desarrollo de software, destacando su relación con el ciclo de vida del desarrollo de software, así como las principales herramientas de gestión que permiten organizar, controlar y garantizar la calidad del diseño de bases de datos.

Transcripción del pódcast. Diseño de estructuras de datos y planificación en el desarrollo de software

Mujer: Durante ocho meses un equipo de software estuvo trabajando en un proyecto. Finalmente, el sistema estaba listo para salir a producción.

Hombre: Comenzaron las pruebas y todo parecía marchar bien... hasta que descubrieron que la base de datos no respondía a las necesidades reales del negocio.

Mujer: En ese momento tuvieron que devolver buena parte del trabajo. Y aunque la tecnología no tenía ningún problema, la falta de planeación terminó pasándoles una factura muy cara.

Hombre: Revisemos otro caso. Un equipo que debe desarrollar el sistema para una clínica. Aunque todos quieren empezar a programar, primero necesitan tener claro qué necesita el sistema y cómo se va a organizar el proyecto.

Mujer: Cuando el equipo se sienta frente al computador, ya el proyecto debe tener una ruta clara, las responsabilidades definidas y las actividades organizadas.

Hombre: Por eso el diseño de estructuras de datos hace parte del Ciclo de Vida del Desarrollo de Software. En ese proceso se entienden las necesidades del negocio, se analizan los requisitos y se define el camino que seguirá el desarrollo.

Mujer: Si esa planeación falla, el proyecto empieza a devolverse. Aparecen cambios de última hora, los tiempos se alargan y los costos terminan creciendo más de la cuenta.

Hombre: Por eso los proyectos no se manejan a punta de memoria. Existen herramientas que permiten dividir el trabajo, asignar responsabilidades y seguir cada etapa del proceso.

Mujer: Una herramienta muy valiosa es la Estructura de Desglose del Trabajo, porque permite convertir un proyecto grande en actividades más fáciles de organizar y controlar.

Hombre: Pero no todo puede hacerse al mismo tiempo. Hay tareas que dependen de otras. Si una se retrasa, las que vienen detrás también tienen que esperar.

Mujer: Con toda la información organizada, el cronograma se convierte en una radiografía del proyecto. Con una sola mirada, el equipo sabe qué avanza, qué se retrasó y qué necesita corregirse.

Hombre: Si cada integrante conoce su responsabilidad, el trabajo fluye mejor. Unos analizan las necesidades del negocio, otros diseñan la base de datos y luego alguien se encarga de administrarla y mantenerla.

Mujer: Esa coordinación evita reprocesos, reduce errores y permite que la base de datos responda a lo que el negocio realmente necesita.

Hombre: Un proyecto con una buena planeación desde el comienzo le facilita al equipo tomar mejores decisiones.

Mujer: Al final, la base de datos termina pareciéndose al proyecto que la construyó. Si el proyecto nació desordenado, difícilmente la base de datos funcionará mejor.

2.1. Metodologías de planificación y el ciclo de vida del desarrollo de software (CVDS)

El diseño de una base de datos relacional no debe abordarse como una actividad aislada. Por el contrario, forma parte del Ciclo de Vida del Desarrollo de Software (CVDS), donde cada decisión tomada durante la planificación influye directamente en la calidad, la escalabilidad y el mantenimiento de la solución tecnológica. La forma de integrar el modelado de datos dentro del CVDS depende de la metodología de gestión de proyectos adoptada por la organización. Cada enfoque presenta características particulares que influyen en la planificación, el diseño y la evolución de la base de datos:

- **Metodologías tradicionales o en cascada.** La planificación y el diseño de la base de datos se realizan de forma secuencial durante las primeras fases del proyecto. Antes de iniciar el diseño conceptual y lógico, los requisitos del cliente deben estar completamente definidos. Este enfoque favorece una amplia documentación y reduce la probabilidad de errores estructurales en

etapas posteriores, aunque ofrece menor flexibilidad frente a cambios en los requerimientos.

- **Metodologías ágiles (Scrum y Kanban).** El diseño de la base de datos evoluciona de manera incremental durante cada ciclo de desarrollo o Sprint. La estructura de almacenamiento se ajusta conforme se priorizan nuevas historias de usuario. Este enfoque facilita la adaptación a cambios del negocio, pero exige aplicar prácticas de refactorización que permitan evolucionar la base de datos sin afectar la integridad de la información almacenada.

Independientemente de la metodología utilizada, el diseño de una base de datos debe desarrollarse mediante tres niveles de abstracción que permiten transformar gradualmente los requisitos del negocio en una solución técnica:

- **Diseño conceptual.** Representa la realidad del negocio mediante entidades, atributos y relaciones, sin depender de un SGBD, del software o del hardware.
- **Diseño lógico.** Transforma el modelo conceptual en un modelo relacional compuesto por tablas, llaves y restricciones, manteniendo la independencia respecto al motor de base de datos que se utilizará.
- **Diseño físico.** Implementa el modelo lógico en un SGBD específico, definiendo aspectos como tipos de datos, índices, almacenamiento y configuraciones orientadas al rendimiento.

La ejecución ordenada de estas etapas permite construir bases de datos consistentes, escalables y fáciles de mantener. Omitir alguna de ellas e iniciar directamente con la creación de tablas en un SGBD constituye una práctica inadecuada

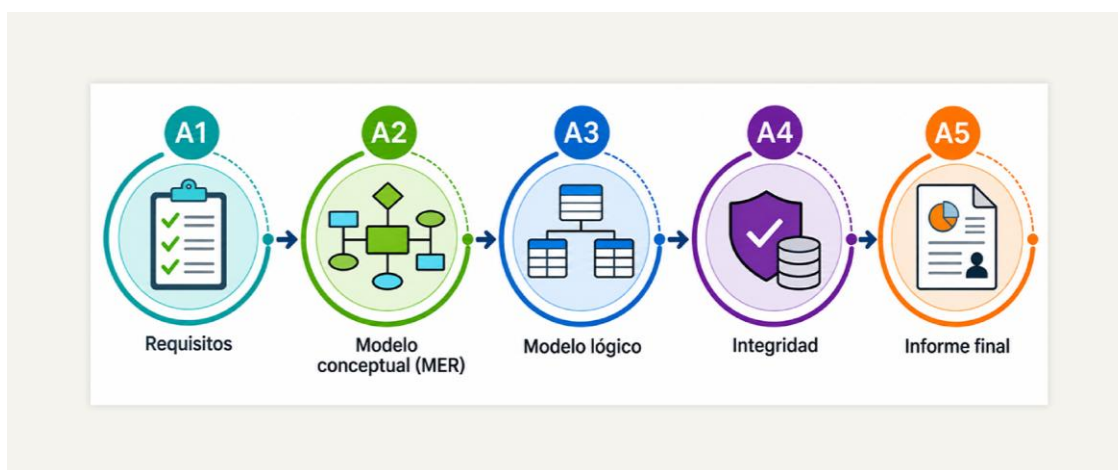
de ingeniería, ya que incrementa la probabilidad de generar redundancias, inconsistencias y problemas de rendimiento durante la operación del sistema.

2.2. Elaboración técnica del plan de trabajo: actividades, secuenciación y dependencias

Uno de los primeros resultados de aprendizaje de este componente formativo consiste en desarrollar la capacidad para elaborar un plan de trabajo técnico que permita organizar las actividades del proyecto de manera estructurada. En el entorno productivo, este documento constituye la hoja de ruta que orienta el desarrollo de la base de datos, facilita el seguimiento de las tareas y contribuye al cumplimiento de los tiempos y entregables acordados con el cliente o con la organización.

La elaboración del plan de trabajo comienza con la descomposición del proceso de diseño en actividades específicas, medibles y organizadas cronológicamente. Esta secuencia permite establecer dependencias entre las tareas y garantizar que cada etapa se inicie únicamente cuando la anterior haya finalizado satisfactoriamente.

Figura 2. Secuencia general del proceso de diseño



La siguiente estructura ejemplifica una Estructura de Desglose del Trabajo (EDT) para el diseño de una base de datos relacional.

Tabla 1. Estructura de Desglose del Trabajo (EDT) para el diseño de estructuras de datos

ID	Actividad	Descripción	Dependencia
A1	Levantamiento y análisis de requisitos	Recolección y análisis de la información mediante entrevistas, revisión documental y otras técnicas para identificar las reglas de negocio que orientarán el diseño de la base de datos.	Ninguna
A2	Construcción del modelo conceptual	Elaboración del Modelo Entidad-Relación (MER) o del Modelo Entidad-Relación Extendido (MERE), con base en los requisitos identificados durante la etapa anterior.	A1 (Final a Inicio)
A3	Diseño del modelo lógico	Transformación del modelo conceptual en un esquema relacional de tablas, aplicando las reglas de transformación y el proceso de normalización.	A2 (Final a Inicio)
A4	Definición de restricciones e integridad	Especificación de tipos de datos, dominios, llaves primarias, llaves foráneas y demás restricciones que garantizan la integridad de la información.	A3 (Final a Inicio)
A5	Validación y documentación	Revisión técnica del diseño, verificación de su consistencia y elaboración del informe final del proyecto.	A4 (Final a Inicio)

La relación de dependencia entre las actividades determina el orden en que deben ejecutarse las tareas del proyecto. Por ejemplo, la actividad A3 depende de la finalización de la actividad A2 mediante una relación Final a Inicio. Esto significa que el

diseño del modelo lógico solo puede comenzar cuando el modelo conceptual ha sido elaborado, revisado y aprobado.

Definir correctamente estas dependencias permite controlar la secuencia del proyecto, reducir reprocesos y optimizar el uso de los recursos disponibles. Además, facilita el seguimiento del cronograma y contribuye a que el diseño de la base de datos avance de forma organizada y coherente con las reglas de negocio establecidas.

2.3. Herramientas de gestión y diagramas de planificación: Gantt y estimación de recursos

Una vez definidas las actividades, la secuencia de ejecución y las dependencias del proyecto, el siguiente paso consiste en planificar su desarrollo mediante herramientas de gestión. Estas aplicaciones permiten organizar las tareas, asignar responsables, controlar los tiempos de ejecución y realizar seguimiento al avance del proyecto durante todo su ciclo de vida.

Uno de los recursos más utilizados para este propósito es el diagrama de Gantt, una representación gráfica que distribuye las actividades del proyecto a lo largo de una línea de tiempo. Este diagrama facilita la planificación del cronograma, la identificación de dependencias entre tareas y el seguimiento del cumplimiento de los plazos establecidos.

Además de representar visualmente el cronograma, las herramientas de gestión permiten centralizar la información del proyecto en entornos colaborativos. Al registrar las actividades de la Estructura de Desglose del Trabajo (EDT), es posible asignar responsables, establecer fechas de inicio y finalización, controlar la disponibilidad de los

recursos y realizar ajustes cuando se presentan cambios durante el desarrollo del proyecto.

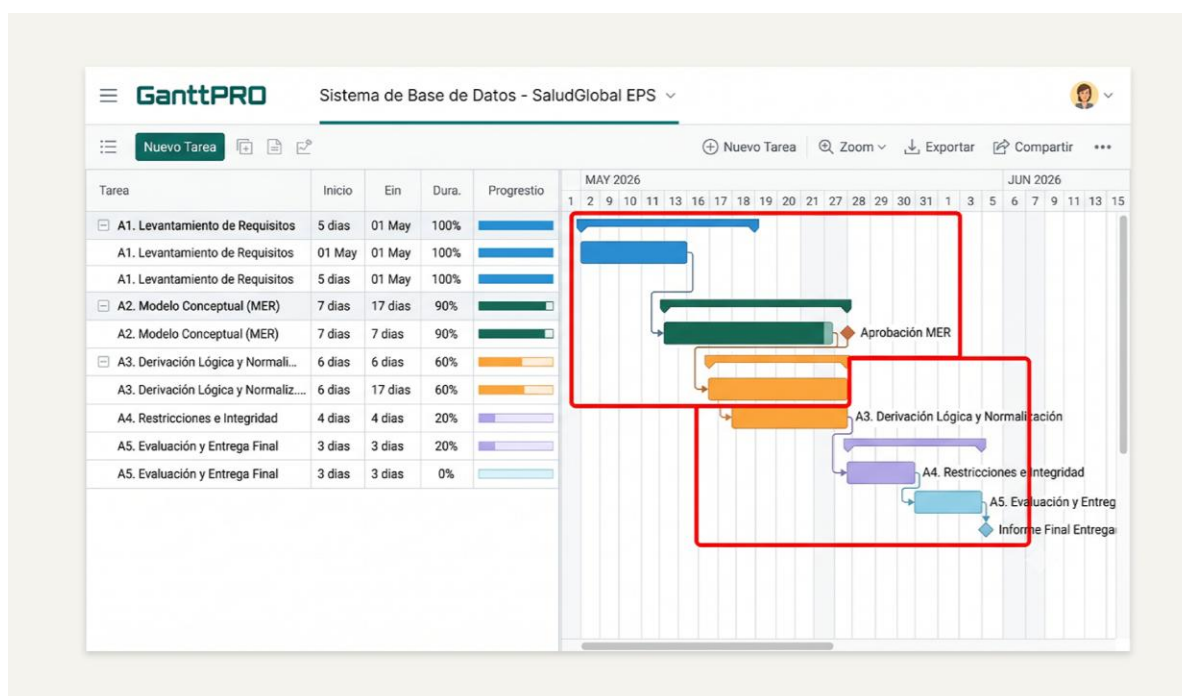
Entre las funcionalidades más utilizadas se encuentran la generación automática de diagramas de Gantt, el cálculo de dependencias, la identificación de la ruta crítica, el seguimiento del avance de las actividades y la administración de la carga de trabajo del equipo. Estas características facilitan la toma de decisiones y contribuyen al cumplimiento de los objetivos del proyecto. Las siguientes herramientas son ampliamente utilizadas para la gestión de proyectos y la planificación de actividades en entornos académicos y empresariales:

- **Microsoft Project.** Solución especializada para la gestión de proyectos de gran complejidad. Permite elaborar cronogramas, administrar recursos, gestionar presupuestos, establecer dependencias entre actividades e identificar la ruta crítica del proyecto.
- **Asana.** Plataforma de trabajo colaborativo que facilita la planificación y el seguimiento de actividades mediante cronogramas, asignación de responsables, control de fechas y comunicación entre los integrantes del equipo.
- **Trello.** Herramienta basada en tableros Kanban que, mediante complementos (Power-Ups), incorpora diagramas de Gantt y funciones de planificación. Su interfaz sencilla favorece la organización de proyectos pequeños y medianos.
- **ClickUp.** Plataforma integral para la gestión de proyectos que incluye diagramas de Gantt, control del tiempo, administración de recursos, seguimiento del avance y documentación centralizada del proyecto.

- **GanttPRO.** Aplicación especializada en la creación y administración de diagramas de Gantt. Permite planificar actividades, gestionar hitos, controlar recursos y generar reportes para el seguimiento del proyecto.
- **Monday.com.** Plataforma colaborativa que integra cronogramas, diagramas de Gantt, automatización de tareas, seguimiento de proyectos y control del trabajo en equipo mediante una interfaz altamente configurable.

La selección de la herramienta depende del tamaño del proyecto, la metodología de trabajo adoptada, la cantidad de integrantes del equipo y las necesidades de la organización. Independientemente de la plataforma utilizada, el objetivo es facilitar la planificación, optimizar la asignación de recursos y garantizar el cumplimiento de los tiempos establecidos durante el diseño y desarrollo de la base de datos.

Figura 3. Ejemplo Gantt en línea



Además de facilitar la programación de actividades, las herramientas de gestión de proyectos proporcionan indicadores que permiten controlar el avance del cronograma y anticipar posibles retrasos. Entre los conceptos más importantes para la planificación se encuentran la holgura y la ruta crítica, ya que ambos permiten identificar las actividades que ofrecen flexibilidad y aquellas que determinan la duración total del proyecto.

- **Holgura.** Corresponde al tiempo máximo que una actividad puede retrasarse sin afectar la fecha de finalización del proyecto. Cuando una tarea dispone de holgura, el director del proyecto puede redistribuir recursos humanos o tecnológicos para atender imprevistos sin comprometer el cronograma general.
- **Ruta crítica (CPM, Critical Path Method).** Es la secuencia de actividades que determina la duración mínima del proyecto. Las tareas que pertenecen a la ruta crítica tienen una holgura igual a cero; por tanto, cualquier retraso en una de ellas ocasiona un aplazamiento en la fecha de finalización del proyecto.

Además de planificar el cronograma, la gestión de proyectos requiere identificar y asignar los recursos necesarios para ejecutar cada actividad. Una adecuada estimación de estos recursos permite distribuir responsabilidades, optimizar el trabajo del equipo y garantizar la disponibilidad de la infraestructura tecnológica durante el desarrollo del proyecto.

- **Recursos humanos.** Corresponden a los perfiles profesionales que participan en el proyecto. Entre ellos se encuentran los ingenieros de requisitos, responsables del levantamiento de información; los diseñadores de datos,

encargados del modelado conceptual y lógico; los administradores de bases de datos (DBA), quienes implementan y administran la base de datos; y los auditores de calidad, encargados de verificar el cumplimiento de los criterios técnicos establecidos.

- **Recursos tecnológicos.** Incluyen la infraestructura de hardware, los Sistemas de Gestión de Bases de Datos (SGBD), las herramientas CASE para el modelado de datos, los sistemas de control de versiones y las demás aplicaciones necesarias para desarrollar, documentar y administrar el proyecto.

La estimación adecuada del tiempo y de los recursos constituye un elemento esencial para la planificación de proyectos de bases de datos. Su correcta aplicación permite reducir riesgos, optimizar la ejecución de las actividades y facilitar el cumplimiento de los objetivos definidos durante el proceso de desarrollo.

3. Modelado conceptual de datos: componentes esenciales del Modelo Entidad-Relación (MER)

El modelado conceptual constituye una de las etapas más importantes en el diseño de bases de datos relacionales, ya que permite representar la realidad del negocio antes de iniciar la implementación técnica de la solución. Durante esta fase, los requisitos recopilados se transforman en un modelo que describe las entidades, sus atributos y las relaciones existentes entre ellas, sin depender de un Sistema de Gestión de Bases de Datos (SGBD) o de aspectos relacionados con el hardware y el software.

El Modelo Entidad-Relación (MER), propuesto por Peter Chen en 1976, es uno de los modelos conceptuales más utilizados para representar la información de una organización. Su propósito consiste en estructurar de manera gráfica los elementos que intervienen en un sistema de información, facilitando la comunicación entre los expertos del negocio y los diseñadores de bases de datos antes de iniciar el diseño lógico y físico.

3.1. Teoría de conjuntos y conceptualización de entidades fuertes y débiles

A continuación, se presenta un video sobre el concepto de entidad en el modelado conceptual, la diferencia entre entidad e instancia, y la clasificación de las entidades en fuertes y débiles, como elementos fundamentales para el diseño de bases de datos.

Video 1. Entidades fuertes y débiles en el modelado conceptual



[Enlace de reproducción del video](#)

Transcripción del video. Entidades fuertes y débiles en el modelado conceptual

En el modelado conceptual, una entidad representa cualquier objeto, persona, lugar, concepto o evento sobre el cual una organización necesita almacenar información dentro de un sistema.

Desde la perspectiva de la teoría de conjuntos, una entidad corresponde a un conjunto de elementos que comparten características comunes dentro de un sistema de información. Por ejemplo, Cliente, Producto, Factura o Curso representan entidades que agrupan elementos con atributos similares.

Es importante diferenciar una entidad de una instancia. La entidad representa la categoría general, mientras que la instancia corresponde a un elemento específico que pertenece a esa categoría.

Transcripción del video. Entidades fuertes y débiles en el modelado conceptual

Por ejemplo, Producto es una entidad, mientras que un artículo identificado como Filtro de aire XY-123 constituye una instancia de esa entidad.

Cada instancia debe contar con una identificación única, que garantice la integridad, la consistencia y la adecuada administración de la información dentro de la base de datos.

Según su forma de identificación, las entidades se clasifican en fuertes y débiles. Una entidad fuerte tiene existencia propia y puede identificarse mediante sus propios atributos, sin depender de otra entidad. Algunos ejemplos son Aprendiz, Instructor, Cliente y Producto

En cambio, una entidad débil depende de una entidad fuerte para existir o identificarse de manera única. Por ello, su identificación incorpora la clave de la entidad de la cual depende. Ejemplos de ello son Detalle de factura respecto a Factura, Línea de pedido respecto a Pedido y Movimiento asociado a una Cuenta bancaria.

Comprender la diferencia entre entidades fuertes y entidades débiles es fundamental para construir modelos de datos consistentes, establecer relaciones adecuadas y diseñar bases de datos eficientes, confiables y alineadas con las necesidades de la organización.

La diferencia entre ambos tipos de entidades puede comprenderse mediante el siguiente ejemplo:

- **Entidad fuerte: cuenta bancaria.** Puede identificarse mediante un número de cuenta único y existe de forma independiente dentro del sistema financiero.
- **Entidad débil: movimiento.** Representa operaciones como depósitos, retiros o transferencias. Su identificación depende de la cuenta bancaria a la que pertenece, ya que un movimiento no tiene significado si no está asociado a una cuenta específica.

La correcta identificación de las entidades fuertes y débiles constituye uno de los primeros pasos en el modelado conceptual, ya que permite establecer adecuadamente las relaciones entre los elementos del sistema y definir posteriormente las claves que garantizarán la integridad de la información durante el diseño lógico de la base de datos.

3.2. Taxonomía exhaustiva de atributos: simples, compuestos, multivaluados y derivados

Una entidad adquiere significado dentro del Modelo Entidad-Relación (MER) cuando se describe mediante atributos, los cuales representan las características que permiten identificarla y registrar la información necesaria para el funcionamiento de la organización. Cada atributo almacena un dato específico asociado a una instancia de la entidad y contribuye a representar la realidad del negocio de manera precisa.

La correcta identificación y clasificación de los atributos permite construir modelos conceptuales más completos, facilita la transformación al modelo lógico y reduce problemas relacionados con la redundancia, la pérdida de información y las dificultades durante las consultas. De acuerdo con su estructura y comportamiento, los atributos pueden clasificarse de la siguiente manera.

- **Atributo simple o atómico.** Representa una única característica que no puede dividirse en elementos con significado propio dentro del contexto del sistema. Algunos ejemplos son la edad de un cliente, el salario de un empleado o el código ISBN de un libro.
- **Atributo compuesto.** Está conformado por varios atributos relacionados que, en conjunto, describen una misma característica. Por ejemplo, una dirección puede descomponerse en tipo de vía, número, barrio, ciudad y departamento. Esta estructura facilita el almacenamiento, la búsqueda y el análisis de la información.

Además de su estructura, los atributos pueden clasificarse según la cantidad de valores que admiten para cada instancia:

- **Atributo monovaluado.** Solo admite un valor para cada instancia de la entidad. Por ejemplo, una persona registra un único número de documento de identidad dentro del sistema.
- **Atributo multivaluado.** Puede contener varios valores asociados a una misma instancia. Algunos ejemplos son los números telefónicos, los correos electrónicos, las especialidades o los títulos académicos de un instructor. Este tipo de atributo requiere un tratamiento especial durante el diseño lógico para evitar redundancia y preservar la integridad de la información.

Otra clasificación importante corresponde a los atributos derivados, cuyo valor se obtiene a partir de otros datos almacenados en la base de datos.

Atributo derivado

Su valor no se almacena de forma permanente, sino que se calcula a partir de otros atributos o de información proporcionada por el sistema. Un ejemplo es la edad, que puede obtenerse a partir de la fecha de nacimiento y de la fecha actual. De esta manera, se evita almacenar datos que cambian con el tiempo y se garantiza que la información consultada sea siempre actualizada.

La identificación adecuada de los diferentes tipos de atributos constituye un paso fundamental en el modelado conceptual. Una correcta clasificación permite representar con mayor precisión la realidad del negocio, facilita la transformación al modelo lógico y contribuye al diseño de bases de datos más consistentes, flexibles y fáciles de mantener.

3.3. Teoría formal de claves para la integridad semántica de los datos

En el Modelo Entidad-Relación (MER), cada instancia de una entidad debe poder identificarse de manera única para evitar duplicidades y garantizar la integridad de la información. Para lograrlo, el modelado conceptual emplea un conjunto de reglas conocidas como teoría de claves, las cuales permiten establecer mecanismos de identificación que posteriormente serán implementados en el modelo lógico de la base de datos.

La correcta selección de las claves facilita el control de la información, fortalece las relaciones entre las entidades y asegura que cada registro pueda diferenciarse de los demás. En el proceso de modelado conceptual se distinguen tres tipos principales de claves:

- **Superclave.** Es un conjunto formado por uno o más atributos que permite identificar de manera única cada instancia de una entidad. Aunque garantiza la identificación, puede contener atributos adicionales que no son necesarios para distinguir los registros. Por ejemplo, en la entidad Vehículo, el conjunto {Placa, Número de motor, Color, Modelo} constituye una superclave, ya que identifica de forma única cada vehículo, aunque incluye atributos redundantes para este propósito.
- **Clave candidata.** Corresponde a una superclave mínima, es decir, contiene únicamente los atributos indispensables para identificar una instancia de forma única. Ninguno de sus atributos puede eliminarse sin perder la capacidad de identificación. En la entidad Vehículo, tanto {Placa} como {Número de motor} representan claves candidatas, ya que cada uno identifica por sí solo un vehículo.
- **Clave primaria (PK, Primary Key)**
- **Clave primaria (PK, <i>Primary Key</i>).** Es la clave candidata seleccionada para identificar oficialmente cada registro de una entidad. Debe cumplir dos condiciones fundamentales: unicidad, de modo que no existan dos registros con el mismo valor, y restricción de no nulos, lo que implica que todos los registros deben contener obligatoriamente un valor para este atributo. En el ejemplo de la entidad Vehículo, el atributo Placa puede seleccionarse como clave primaria para identificar cada vehículo y establecer relaciones con otras entidades del modelo.

La selección adecuada de la clave primaria constituye una decisión fundamental durante el diseño de una base de datos, ya que permite garantizar la integridad de los

registros y facilita el establecimiento de relaciones entre las diferentes entidades que conforman el modelo relacional.

3.4. Restricción de dominio de un atributo

Además de identificar correctamente las entidades, los atributos y las claves, el modelado conceptual debe definir el dominio de cada atributo. El dominio corresponde al conjunto de valores válidos que un atributo puede almacenar y constituye un mecanismo para preservar la calidad y la consistencia de la información desde las primeras etapas del diseño.

Definir dominios adecuados permite prevenir errores durante el registro de datos y facilita la implementación posterior de reglas de validación en la base de datos. Por ejemplo, el atributo Salario de la entidad Trabajador puede definirse como un número real positivo, lo que impide registrar valores negativos o incompatibles con las reglas del negocio.

Caso de uso: modelado conceptual de la entidad Paciente en SaludGlobal EPS

Durante el levantamiento de requisitos, la organización SaludGlobal EPS identifica la necesidad de administrar la información clínica y demográfica de sus pacientes. Entre los requerimientos definidos se encuentran el almacenamiento de la dirección detallada para apoyar la distribución de medicamentos, el registro de múltiples números telefónicos de contacto y el cálculo automático de la edad a partir de la fecha de nacimiento. Con base en estos requerimientos, el equipo de diseño elabora el siguiente modelo conceptual para la entidad Paciente.

Tabla 2. Modelado conceptual de la entidad Paciente

Componente MER	Identificación	Justificación	Clasificación
Entidad	Paciente	Representa el conjunto de personas que reciben servicios de salud y sobre las cuales se administra información clínica y administrativa.	Entidad fuerte
Atributo	Documento de identidad	Identifica de forma única a cada paciente dentro del sistema.	Clave primaria (PK), atributo simple y monovaluado
Atributo	Dirección de contacto	Permite registrar información detallada de ubicación, como tipo de vía, número, barrio y ciudad, necesaria para procesos de georreferenciación y distribución de medicamentos.	Atributo compuesto
Atributo	Números telefónicos	Permite almacenar varios números de contacto asociados a un mismo paciente.	Atributo multivaluado
Atributo	Fecha de nacimiento	Registra la fecha de nacimiento del paciente y sirve como dato base para otros cálculos.	Atributo simple y monovaluado
Atributo	Edad	Se obtiene automáticamente a partir de la fecha de nacimiento y de la fecha actual, evitando almacenar un dato que cambia con el tiempo.	Atributo derivado
Dominio	Estado de afiliación	Restringe los valores permitidos a Activo, Inactivo, En mora y Fallecido, garantizando la consistencia de la información registrada.	Restricción de dominio

Este ejemplo integra varios de los conceptos fundamentales del Modelo Entidad-Relación. La entidad Paciente corresponde a una entidad fuerte; el Documento de identidad actúa como clave primaria; la Dirección de contacto representa un atributo compuesto; los Números telefónicos corresponden a un atributo multivaluado; la Edad se modela como un atributo derivado, y el Estado de afiliación incorpora una restricción de dominio que limita los valores permitidos. En conjunto, estas decisiones contribuyen a construir un modelo conceptual consistente, preparado para su transformación al modelo lógico y a la implementación posterior de la base de datos.

4. Semántica avanzada y extensiones en el modelado conceptual (MERE)

El modelado conceptual de bases de datos no se limita a identificar entidades y atributos. También requiere representar las relaciones existentes entre los diferentes elementos del sistema y las reglas de negocio que determinan su comportamiento. A medida que aumenta la complejidad de los procesos organizacionales, resulta necesario incorporar mecanismos que permitan expresar con mayor precisión las asociaciones, restricciones y jerarquías presentes en la información.

El Modelo Entidad-Relación Extendido (MERE) amplía las capacidades del Modelo Entidad-Relación (MER) al incorporar conceptos como la generalización, la especialización, la agregación y otras abstracciones que facilitan la representación de escenarios más complejos. Estas extensiones permiten construir modelos conceptuales más cercanos a la realidad del negocio y simplifican su posterior transformación al modelo lógico.

4.1. El concepto de relación: grado, cardinalidad y restricciones de participación

Una relación representa la asociación lógica entre dos o más entidades dentro de un modelo conceptual. Gracias a estas asociaciones es posible representar la forma en que interactúan los diferentes elementos de un sistema de información y reflejar las reglas de negocio que rigen el funcionamiento de una organización. Para describir correctamente una relación, es necesario analizar tres aspectos fundamentales: el grado, la cardinalidad y las restricciones de participación. Estos elementos permiten definir cómo se conectan las entidades y cuáles son las condiciones que regulan dicha interacción.

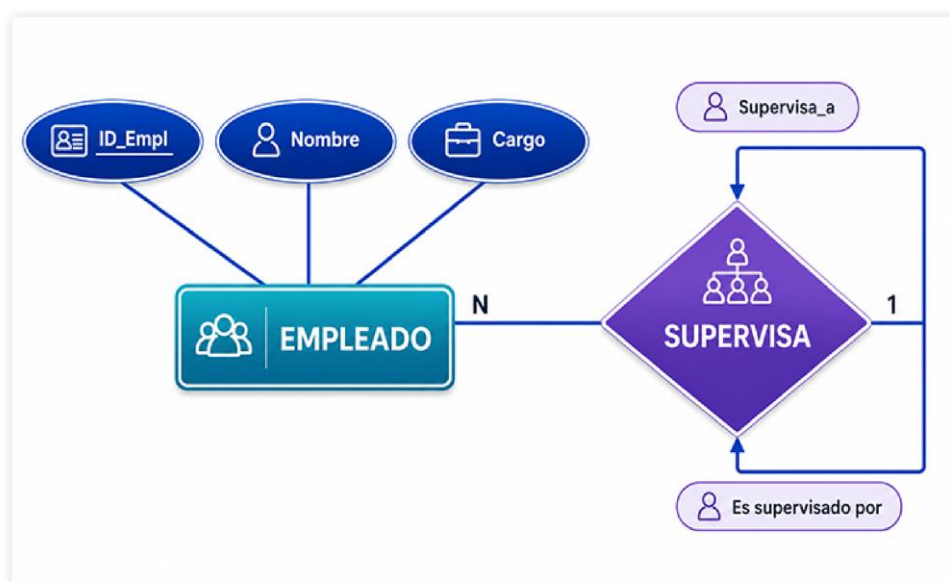
Grado de una relación

El grado corresponde al número de entidades que participan en una relación. De acuerdo con esta característica, las relaciones pueden clasificarse de la siguiente manera.

Unaria o recursiva

Una entidad se relaciona consigo misma. Por ejemplo, en la entidad Empleado, la relación Supervisa permite representar que un empleado puede supervisar a otros empleados de la misma organización.

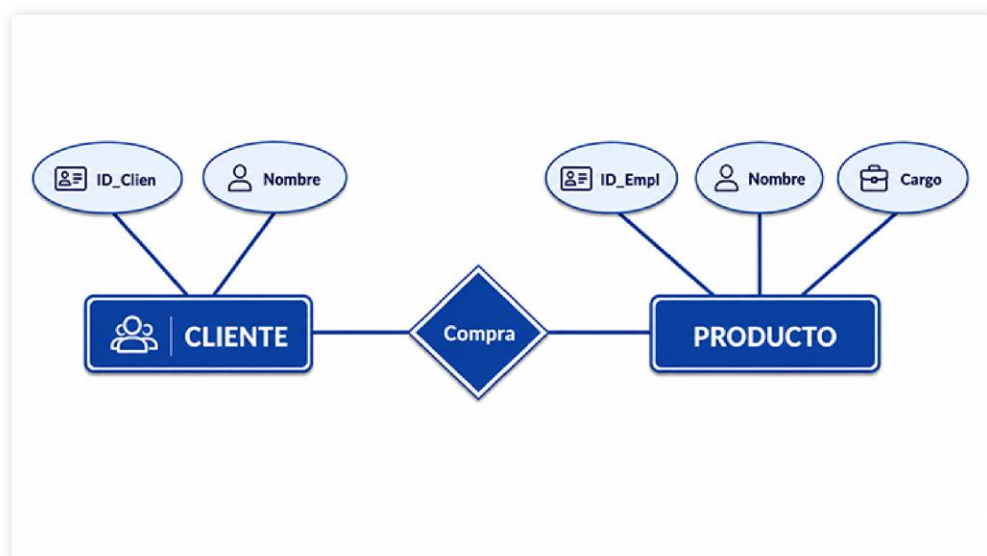
Figura 4. Ejemplo relación recursiva (unaria) de supervisión



Binaria

Involucra exactamente dos entidades y constituye el tipo de relación más utilizado en el diseño de bases de datos. Un ejemplo es la relación entre las entidades Cliente y Producto, donde un cliente realiza la compra de uno o varios productos.

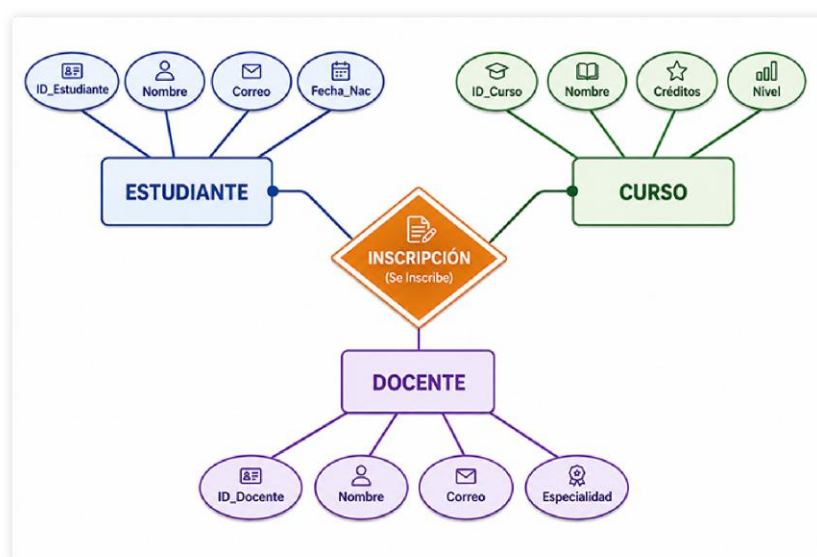
Figura 5. Ejemplo relación binaria entre cliente y producto



Ternaria o de grado superior

Involucra tres o más entidades dentro de una misma relación. Su utilización se justifica cuando la regla de negocio no puede representarse adecuadamente mediante varias relaciones binarias independientes.

Figura 6. Ejemplo relación ternaria: inscripción



La identificación correcta del grado de una relación permite representar con mayor precisión las reglas del negocio y facilita la construcción de modelos conceptuales consistentes. Esta información será determinante durante el diseño lógico, ya que influirá en la forma como las relaciones se transforman posteriormente en tablas y restricciones dentro de la base de datos.

Cardinalidad de mapeo (razones de cardinalidad)

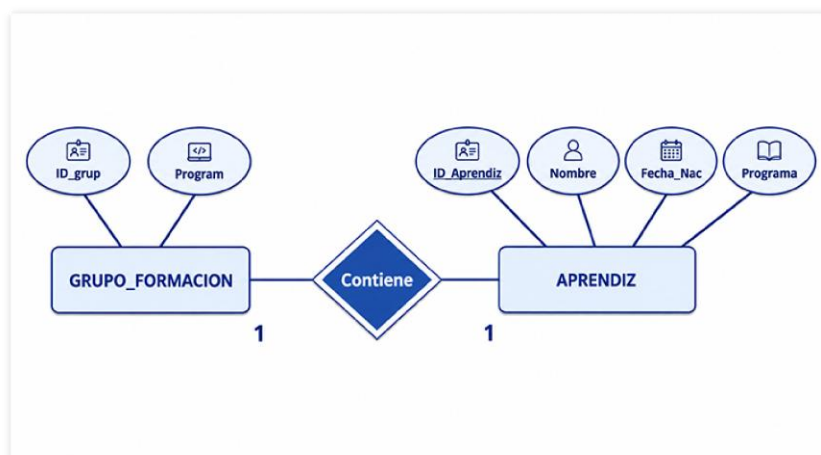
La cardinalidad de mapeo indica el número máximo de instancias de una entidad que pueden asociarse con una instancia de otra entidad mediante una relación. Este concepto permite representar las reglas de negocio y definir cómo interactúan los diferentes elementos del modelo conceptual.

De acuerdo con el número de ocurrencias permitidas, las relaciones pueden clasificarse de la siguiente manera.

Uno a uno (1:1)

Cada instancia de una entidad se relaciona con una única instancia de la otra entidad y viceversa. Un ejemplo es la relación entre Instructor y Centro de formación, cuando cada centro es dirigido por un solo instructor y cada instructor dirige únicamente un centro.

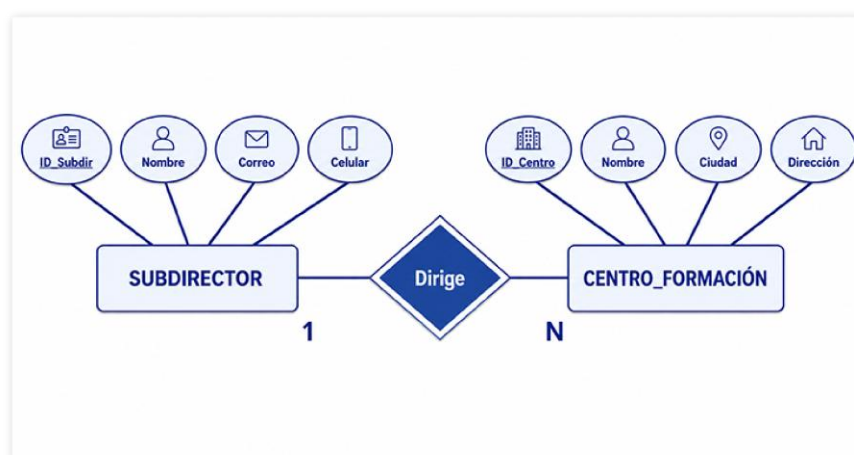
Figura 7. Ejemplo relación uno a uno (1:1)



Uno a varios (1:N)

Una instancia de una entidad puede relacionarse con varias instancias de la otra, mientras que cada instancia de esta última solo puede asociarse con una de la primera. Por ejemplo, un Grupo de formación puede estar conformado por varios Aprendices, pero cada aprendiz pertenece a un solo grupo.

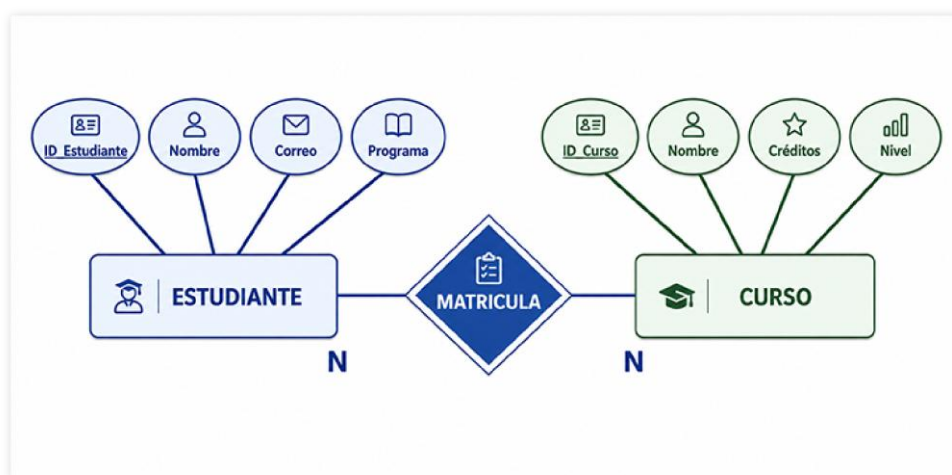
Figura 8. Ejemplo relación uno a varios (1:N)



Varios a varios (N:M)

Varias instancias de una entidad pueden relacionarse con varias instancias de la otra. Por ejemplo, un Aprendiz puede desarrollar varios Resultados de aprendizaje, y un mismo resultado puede estar asociado a diferentes aprendices. En este tipo de relación es común que existan atributos propios de la asociación, como la calificación final, la fecha de evaluación o el porcentaje de asistencia, los cuales describen la relación y no a las entidades de forma independiente.

Figura 9. Ejemplo relación varios a varios (N:M)



La correcta identificación de la cardinalidad facilita la representación de las reglas de negocio y constituye un paso fundamental para la transformación del modelo conceptual en el modelo lógico.

Restricciones de participación (cardinalidad mínima)

Además de establecer el número máximo de asociaciones entre entidades, el modelo conceptual define si la participación de una entidad en una relación es obligatoria u opcional. Esta condición se conoce como restricción de participación o

cardinalidad mínima. Las restricciones de participación pueden clasificarse de la siguiente manera:

- **Participación total u obligatoria.** Todas las instancias de la entidad deben participar al menos una vez en la relación. La cardinalidad mínima es 1. Por ejemplo, todo Envío debe estar asociado obligatoriamente a un Cliente que realiza la solicitud.
- **Participación parcial u opcional.** Las instancias de la entidad pueden participar o no en la relación. La cardinalidad mínima es 0. Por ejemplo, un Cliente puede estar registrado en el sistema sin haber realizado todavía ninguna Compra.

La definición de la cardinalidad y de las restricciones de participación permite representar con mayor precisión las reglas de negocio de una organización. Estos elementos sirven de base para construir modelos conceptuales consistentes y facilitan la implementación posterior de las relaciones y restricciones de integridad en la base de datos relacional.

4.2. Extensiones del Modelo (MERE): jerarquías de generalización y especialización

En algunos sistemas de información, varias entidades comparten características comunes, pero también poseen atributos o comportamientos que las diferencian. En estos casos, el Modelo Entidad-Relación Extendido (MERE) incorpora mecanismos que permiten representar estas similitudes y diferencias sin duplicar información dentro del modelo conceptual.

Las principales extensiones utilizadas para este propósito son la generalización y la especialización, las cuales organizan las entidades mediante jerarquías compuestas por una superclase y una o varias subclases:

- **Generalización.** Es un proceso de diseño ascendente que identifica las características comunes de varias entidades para integrarlas en una entidad más general denominada superclase. Esta estrategia reduce la redundancia y facilita la organización del modelo conceptual.
- **Especialización.** Un proceso de diseño descendente mediante el cual una superclase se divide en varias subclases, de acuerdo con atributos, funciones o características particulares de cada grupo. Cada subclase incorpora propiedades específicas sin perder las características comunes definidas en la superclase.

Una característica fundamental de estas jerarquías es el principio de herencia, mediante el cual las subclases reciben automáticamente la clave primaria, los atributos y las relaciones definidas en la superclase. Gracias a este mecanismo, se evita la duplicación de información y se mantiene la consistencia del modelo conceptual.

Además de la herencia, las jerarquías deben definir reglas que establezcan cómo se distribuyen las instancias entre las diferentes subclases.

- **Disjunta (exclusiva).** Cada instancia de la superclase solo puede pertenecer a una única subclase. Por ejemplo, un Vehículo puede clasificarse como Automóvil o Camión, pero no como ambos al mismo tiempo.
- **Superpuesta (solapada).** Una misma instancia puede pertenecer simultáneamente a varias subclases. Por ejemplo, una Persona puede

desempeñar al mismo tiempo los roles de Estudiante y Empleado dentro de una institución educativa.

Las jerarquías también requieren definir si todas las instancias de la superclase deben pertenecer a alguna subclase.

- **Total.** Todas las instancias de la superclase deben pertenecer obligatoriamente a una de las subclases definidas.
- **Parcial.** Algunas instancias pueden permanecer únicamente en la superclase, sin pertenecer a ninguna subclase específica.

La aplicación adecuada de la generalización y la especialización permite construir modelos conceptuales más organizados, reducir la redundancia de información y representar con mayor precisión la estructura y el comportamiento de los datos dentro de una organización.

4.3. Semántica de la agregación y restricciones de exclusividad en relaciones

En determinadas situaciones, las reglas de negocio requieren representar relaciones de mayor complejidad que no pueden modelarse únicamente mediante entidades y asociaciones tradicionales. Para estos casos, el Modelo Entidad-Relación Extendido incorpora mecanismos como la agregación y las restricciones de exclusividad, los cuales permiten representar escenarios más cercanos a la realidad organizacional.

La agregación consiste en tratar una relación existente entre dos entidades como si fuera una nueva entidad conceptual. Este mecanismo permite establecer nuevas relaciones sobre una asociación previamente definida, evitando estructuras excesivamente complejas y facilitando la representación de procesos empresariales. Por

ejemplo, una relación entre las entidades Empleado y Proyecto, denominada Participa, puede convertirse en una unidad conceptual sobre la cual otra entidad, como Auditoría, registre información relacionada con la participación de los empleados en cada proyecto.

Por otra parte, las restricciones de exclusividad establecen reglas que limitan las relaciones que una entidad puede mantener simultáneamente con otras entidades. Estas restricciones garantizan que el modelo conceptual refleje correctamente las políticas y condiciones establecidas por la organización:

- **Agregación.** Permite representar una relación como una entidad conceptual para establecer nuevas asociaciones con otras entidades del modelo. Esta técnica facilita la representación de procesos complejos sin incrementar innecesariamente el número de relaciones.
- **Exclusividad en relaciones.** Define situaciones en las que una entidad puede participar en varias relaciones posibles, pero únicamente una de ellas puede estar activa en un momento determinado. Estas restricciones sirven como base para implementar posteriormente reglas de integridad dentro de la base de datos.

La utilización de estas extensiones amplía las capacidades del Modelo Entidad-Relación y permite representar estructuras organizacionales más complejas de forma clara y consistente. De esta manera, el modelo conceptual se convierte en una representación más precisa de las reglas de negocio y facilita la transición hacia las etapas posteriores del diseño de la base de datos.

Caso de uso: estructuración avanzada del MERE para LogísticaExpress S. A.

Para integrar los conceptos de generalización, especialización, agregación y restricciones de participación, considere el caso de LogísticaExpress S. A., una empresa dedicada al transporte de mercancías. Durante el levantamiento de requisitos se identificaron varias reglas de negocio que requieren el uso de las extensiones del Modelo Entidad-Relación Extendido (MERE) para representar adecuadamente la realidad operativa de la organización. Entre las principales reglas de negocio se encuentran las siguientes:

- La empresa dispone de una flota de vehículos conformada por camiones y motocicletas. Cada vehículo pertenece exclusivamente a una de estas categorías. Los camiones registran la capacidad de carga y el volumen de almacenamiento, mientras que las motocicletas almacenan el cilindraje y el tipo de baúl.
- Los conductores pueden operar diferentes vehículos a lo largo del tiempo y cada vehículo puede ser conducido por varios conductores. Además, es necesario registrar la fecha de asignación y las horas de conducción correspondientes a cada asignación.
- Cada orden de servicio se asigna a una ruta logística específica y debe asociarse con la combinación de conductor y vehículo responsable del despacho, con el fin de garantizar la trazabilidad de la operación.

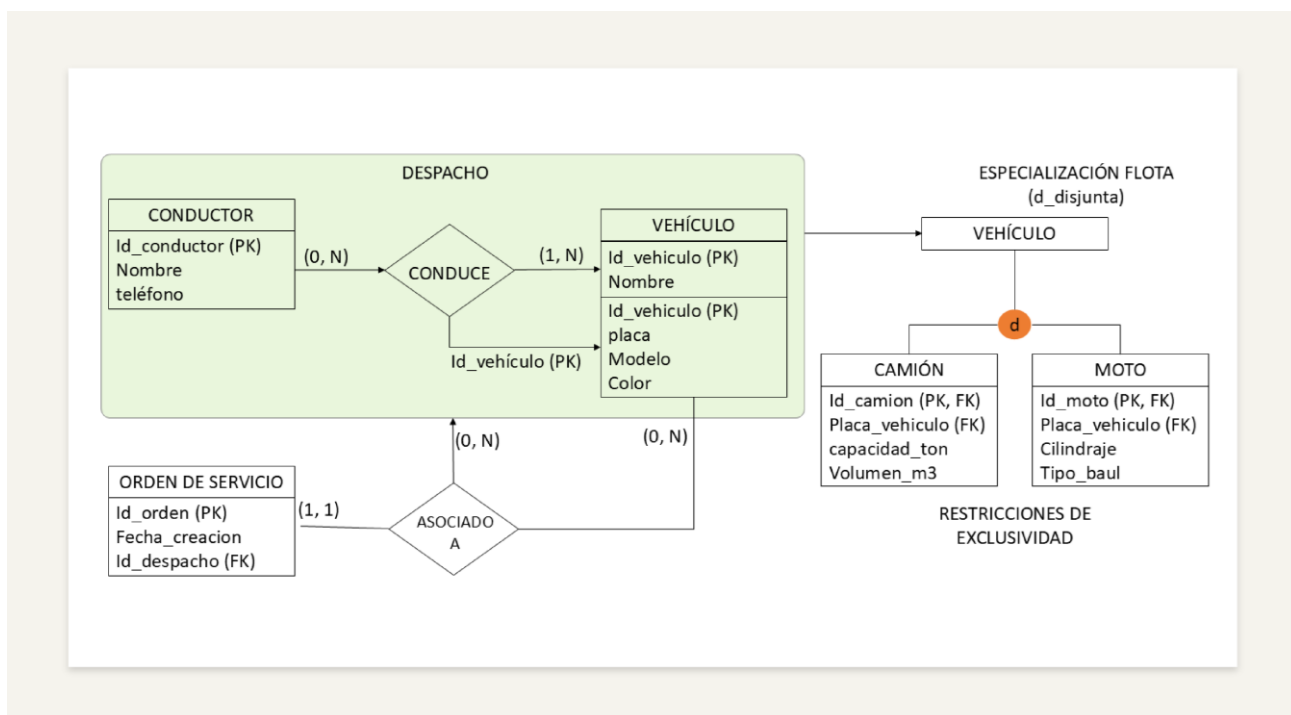
Con base en estas reglas de negocio, el equipo de diseño elabora el siguiente modelo conceptual.

Tabla 3. Aplicación de las extensiones del MERE en LogísticaExpress S. A.

Regla de negocio	Componente del MERE	Aplicación en el modelo
Clasificación de la flota de vehículos.	Especialización total y disyunta	La entidad Vehículo actúa como superclase, mientras que Camión y Motocicleta corresponden a subclases. Ambas heredan la clave primaria y los atributos comunes del vehículo, además de incorporar sus atributos específicos. La especialización es total porque todo vehículo debe pertenecer a una de las subclases, y es disyunta porque un vehículo no puede clasificarse simultáneamente como camión y motocicleta.
Asignación de conductores a vehículos.	Relación varios a varios (N:M)	La relación Conduce vincula las entidades Conductor y Vehículo. Esta relación incorpora atributos propios, como Fecha de asignación y Horas de conducción, los cuales describen la interacción entre ambas entidades y no pertenecen de forma independiente a ninguna de ellas.
Asignación de órdenes de servicio.	Agregación	La relación Conduce, junto con las entidades Conductor y Vehículo, se considera una unidad conceptual sobre la cual se establece la relación con la entidad Orden de servicio. De esta manera, cada orden queda asociada a la combinación específica de conductor y vehículo responsable del transporte, garantizando la trazabilidad de la operación.

Este caso evidencia cómo las extensiones del Modelo Entidad-Relación Extendido permiten representar escenarios empresariales de mayor complejidad. La aplicación de la especialización, las relaciones de cardinalidad varios a varios y la agregación facilitan la construcción de modelos conceptuales más precisos, reducen la ambigüedad en las reglas de negocio y preparan la estructura de datos para su transformación al modelo lógico e implementación en un Sistema de Gestión de Bases de Datos (SGBD).

Figura 10. Modelo E-R Extendido (MERE) del caso de uso



5. Ingeniería de software relacional: metodología de transformación lógica y herramientas CASE

Una vez finalizado el modelo conceptual mediante el Modelo Entidad-Relación Extendido (MERE), el siguiente paso consiste en transformarlo en una estructura que pueda implementarse en un Sistema de Gestión de Bases de Datos (SGBD). Esta etapa corresponde al diseño lógico, cuyo propósito es convertir los elementos del modelo conceptual en tablas, columnas, claves y restricciones propias del modelo relacional.

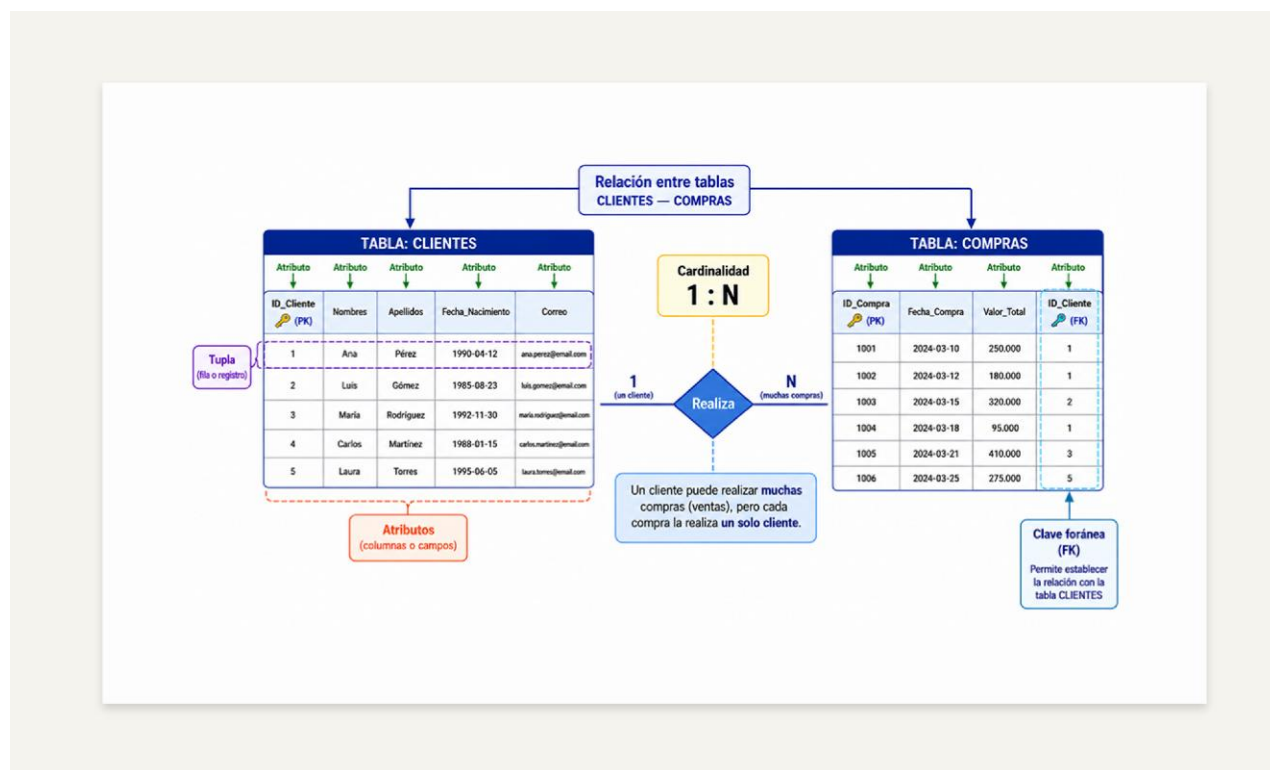
El diseño lógico constituye el puente entre el análisis del negocio y la implementación técnica de la base de datos. Aunque todavía no depende de un SGBD específico, define la estructura que posteriormente será implementada en motores como PostgreSQL, MySQL, Oracle o SQL Server.

Durante esta unidad se estudiarán las reglas de transformación que permiten convertir entidades en tablas, atributos en columnas y relaciones en mecanismos de integridad referencial mediante llaves primarias y llaves foráneas. Asimismo, se abordará el uso de herramientas CASE (Computer-Aided Software Engineering), las cuales apoyan el diseño de bases de datos y automatizan parte del proceso de desarrollo.

5.1. Arquitectura del modelo lógico relacional y la teoría de Codd

El modelo lógico relacional, propuesto por Edgar F. Codd, se fundamenta en la teoría de conjuntos y en la lógica de predicados. Este modelo proporciona una estructura organizada para almacenar y administrar la información mediante tablas relacionadas entre sí. Para comprender el funcionamiento del modelo relacional, es importante familiarizarse con algunos conceptos fundamentales:

Figura 11. Ejemplo elementos fundamentales del modelo relacional



- **Relación (tabla).** Es una estructura bidimensional organizada en filas y columnas que almacena información de una misma entidad. En el modelo relacional, el término relación equivale a una tabla y no debe confundirse con las relaciones del Modelo Entidad-Relación (MER), las cuales representan asociaciones entre entidades.
- **Tupla (fila o registro).** Corresponde a cada una de las filas de una tabla. Cada tupla representa una instancia específica de la entidad almacenada.
- **Atributo (columna o campo).** Corresponde a cada una de las columnas de la tabla. Cada atributo almacena un tipo específico de información y todos sus valores deben pertenecer al mismo dominio de datos.

- **Grado de una relación.** Corresponde al número total de atributos o columnas que conforman una tabla.
- **Cardinalidad de una relación.** Representa el número total de tuplas o registros almacenados en una tabla en un momento determinado.

Además de definir estos conceptos, el modelo relacional establece un conjunto de reglas que garantizan la integridad y la consistencia de la información almacenada.

- **Unicidad de las tuplas.** No pueden existir dos registros completamente idénticos dentro de una misma tabla.
- **Independencia del orden.** El orden en que se almacenan las filas y las columnas no modifica el significado de la información ni afecta el funcionamiento del SGBD.
- **Atomicidad de los valores.** Cada celda de la tabla debe contener un único valor indivisible. No se permite almacenar listas, colecciones o múltiples valores dentro de un mismo atributo.
- **Dominio de los atributos.** Todos los valores almacenados en una columna deben pertenecer al dominio definido para ese atributo y cumplir las restricciones establecidas durante el diseño.

El cumplimiento de estos principios garantiza que las tablas del modelo relacional mantengan una estructura consistente y preparada para implementar mecanismos de integridad referencial, normalización y optimización del almacenamiento. Estas características constituyen la base para el diseño e implementación de bases de datos confiables en entornos organizacionales.

5.2. Reglas de transformación formal del MER/MERE al modelo relacional de tablas

Una vez elaborado el modelo conceptual, el siguiente paso consiste en transformarlo en un modelo lógico que pueda implementarse en un Sistema de Gestión de Bases de Datos (SGBD). Este proceso se realiza mediante un conjunto de reglas de transformación que permiten convertir entidades, atributos y relaciones en tablas, columnas y restricciones de integridad referencial. La aplicación sistemática de estas reglas garantiza que el modelo lógico conserve la estructura y las reglas de negocio definidas durante el modelado conceptual:

- **Regla 1. Transformación de entidades fuertes.** Cada entidad fuerte del MER se convierte en una tabla independiente. La clave primaria de la entidad se transforma en la clave primaria de la tabla y los atributos simples pasan a convertirse en columnas.
- **Regla 2. Transformación de entidades débiles.** Cada entidad débil también se transforma en una tabla. Su clave primaria está compuesta por la clave primaria de la entidad fuerte de la que depende, la cual actúa como llave foránea, junto con la clave parcial que identifica la entidad débil.
- **Regla 3. Transformación de relaciones uno a varios (1:N).** La clave primaria de la entidad ubicada en el lado uno (1) se incorpora como llave foránea en la tabla correspondiente al lado varios (N), estableciendo la relación entre ambas tablas y garantizando la integridad referencial.
- **Regla 4. Transformación de relaciones varios a varios (N:M).** Las relaciones de este tipo requieren la creación de una tabla intermedia o tabla puente. Esta nueva tabla contiene como llaves foráneas las claves primarias de las

dos tablas relacionadas y, cuando la relación posee atributos propios, estos también se incorporan como columnas de la tabla puente.

- **Regla 5. Transformación de atributos multivaluados.** Los atributos multivaluados no pueden almacenarse directamente en una tabla porque el modelo relacional admite un único valor por celda. Por esta razón, se crea una nueva tabla que contiene la clave primaria de la entidad original como llave foránea y el valor correspondiente al atributo multivaluado.

La aplicación correcta de estas reglas permite transformar el modelo conceptual en un esquema relacional consistente y preparado para su implementación en un SGBD. Además, facilita la preservación de la integridad de los datos y garantiza que las relaciones definidas durante el análisis del negocio se mantengan durante la construcción de la base de datos.

5.3. Criterios de selección e Ingeniería de Software Asistida por Computadora (Herramientas CASE)

A medida que aumenta la complejidad del diseño de una base de datos, también se incrementa la necesidad de utilizar herramientas que faciliten el modelado, reduzcan errores y automaticen parte del proceso de desarrollo. Para este propósito se emplean las herramientas CASE (Computer-Aided Software Engineering), aplicaciones especializadas que apoyan el análisis, el diseño y la documentación de sistemas de información.

En el diseño de bases de datos, estas herramientas permiten elaborar modelos conceptuales y lógicos, validar la consistencia de las estructuras, generar código SQL automáticamente y mantener sincronizados los modelos con la implementación física de la base de datos. La selección de una herramienta CASE debe responder a las

necesidades del proyecto y a las características técnicas de la organización. Entre los principales criterios de evaluación se encuentran los siguientes:

- **Soporte de notaciones estándar.** La herramienta debe permitir utilizar notaciones ampliamente reconocidas, como Information Engineering (IE) o IDEF1X, facilitando la interpretación de los modelos por parte de analistas, diseñadores de datos y administradores de bases de datos (DBA).
- **Ingeniería hacia adelante (Forward Engineering).** Permite generar automáticamente el código SQL a partir del modelo lógico. Mediante este proceso, la herramienta crea instrucciones como CREATE TABLE, ALTER TABLE, PRIMARY KEY y FOREIGN KEY, listas para ejecutarse en un Sistema de Gestión de Bases de Datos (SGBD).
- **Ingeniería inversa (Reverse Engineering).** Permite reconstruir el modelo lógico a partir de una base de datos existente. Para ello, la herramienta analiza la estructura almacenada en el SGBD y genera automáticamente el diagrama correspondiente, facilitando las actividades de documentación, mantenimiento y auditoría.

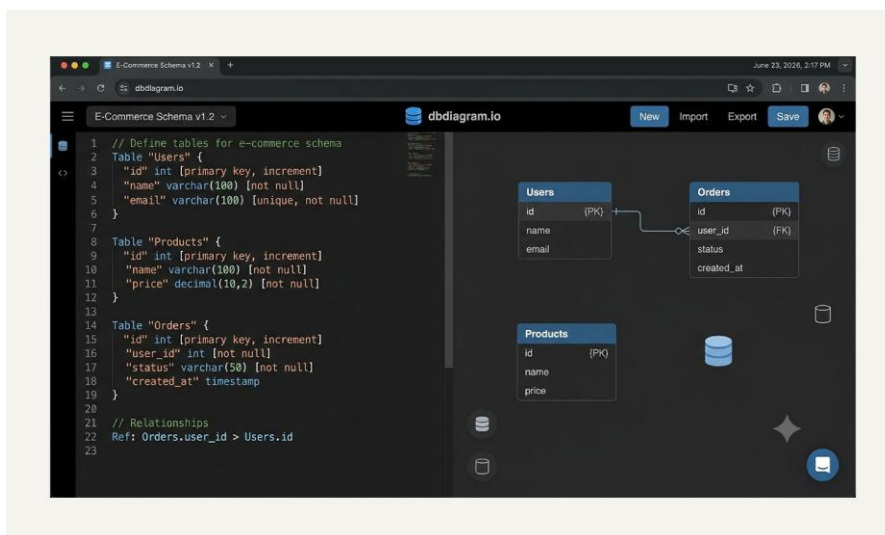
Actualmente existen diversas herramientas CASE que apoyan el diseño de bases de datos y ofrecen funcionalidades orientadas a diferentes tipos de proyectos:

- **dbdiagram.io.** Plataforma web orientada al modelado de bases de datos mediante una sintaxis sencilla basada en texto. Es una alternativa práctica para proyectos académicos y prototipos de rápida elaboración.
- **MySQL Workbench.** Herramienta oficial para el diseño, administración y mantenimiento de bases de datos MySQL. Permite desarrollar modelos, generar código SQL y realizar ingeniería inversa de manera integrada.

- **Erwin Data Modeler.** Solución ampliamente utilizada en entornos empresariales para el modelado conceptual, lógico y físico de bases de datos. Ofrece funciones avanzadas de documentación, estandarización y administración de modelos de gran complejidad.
- **Oracle SQL Developer Data Modeler.** Herramienta gratuita desarrollada por Oracle para el diseño de bases de datos. Permite crear modelos conceptuales, lógicos y físicos, además de realizar ingeniería hacia adelante e ingeniería inversa sobre diferentes Sistemas de Gestión de Bases de Datos.

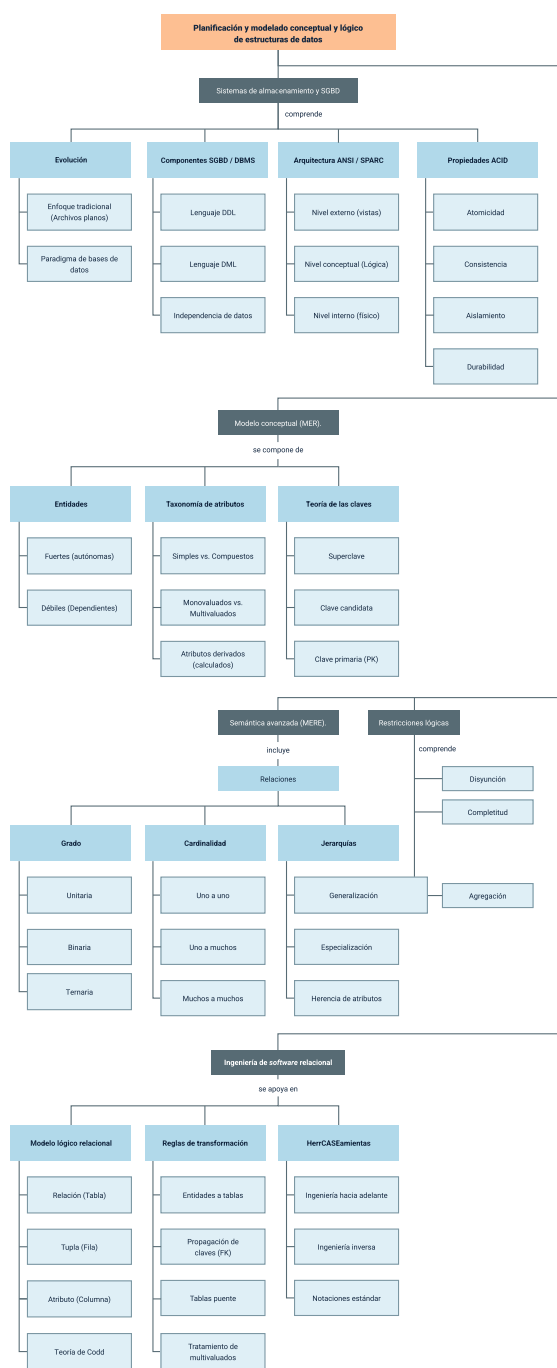
La incorporación de herramientas CASE en el proceso de diseño contribuye a mejorar la calidad de los modelos de datos, reducir errores durante la implementación y optimizar la documentación técnica del proyecto. Asimismo, facilita el trabajo colaborativo entre los diferentes integrantes del equipo de desarrollo y favorece la construcción de bases de datos consistentes, mantenibles y alineadas con las necesidades de la organización.

Figura 12. Pantallazo de herramienta que mezcla utilidades visuales y consola de código integrado



Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Glosario

Atributos: propiedades descriptivas de una entidad o relación que poseen una semántica y un valor específico para cada instancia de la misma.

Clave primaria (PK): identificador único y obligatorio seleccionado por el diseñador para asegurar que cada instancia de una entidad sea distinguible de todas las demás.

DBA: siglas de Database Administrator (Administrador de Bases de Datos), perfil profesional responsable de la gestión, mantenimiento y configuración técnica de los sistemas de datos.

Entidad: cualquier objeto, persona, lugar, concepto o evento del mundo real que posee una existencia independiente y sobre el cual la organización necesita almacenar información.

Esquema de BD: descripción formal y global de la estructura lógica de la base de datos, que actúa como el esqueleto analítico del sistema.

Herramientas CASE: soluciones de software especializado (Computer-Aided Software Engineering) que automatizan la diagramación, transformación lógica y generación de código de bases de datos.

Llave foránea (FK): mecanismo de integridad referencial donde la clave primaria de una tabla se replica en otra para establecer un vínculo lógico entre ambas estructuras.

Metadatos: definidos estrictamente como datos acerca de los datos; son las descripciones lógicas de la estructura almacenadas en el catálogo o diccionario de datos del sistema.

Propiedades ACID: estándar de robustez transaccional que garantiza la invulnerabilidad de la información mediante la Atomicidad, Consistencia, Aislamiento y Durabilidad.

Script: conjunto de instrucciones estructuradas en lenguaje SQL (generalmente DDL) generado de forma automatizada para crear tablas, claves y restricciones en un motor de base de datos.

SGBD: Software de alta ingeniería que actúa como interfaz entre la base de datos física, los usuarios y las aplicaciones, centralizando la administración y seguridad de los datos.

Triggers: denominados también disparadores, son reglas de verificación estricta que se implementan para impedir la corrupción de la lógica del negocio ante acciones específicas.

Tupla de información: constituye cada una de las filas individuales (registros) que componen una relación o tabla, representando una ocurrencia o instancia particular de los datos.

Referencias bibliográficas

Camps Paré, R., Casillas Santillán, L., Costal Costa, D., Ginestà, M., Martín Escofet, C., & Pérez Mora, O. (2024). Software libre. Universitat Oberta de Catalunya.

<https://www.uoc.edu/pdf/masters/oficiales/img/913.pdf>

Casas Roma, J. (2013). Introducción al diseño de bases de datos. Universitat Oberta de Catalunya.

[https://gc.scalahed.com/recursos/files/r161r/w25213w/M1CDN112_S1_Introduccion al diseno de bases.pdf](https://gc.scalahed.com/recursos/files/r161r/w25213w/M1CDN112_S1_Introduccion_al_diseno_de_bases.pdf)

CEPETEL. (2023). Introducción a bases de datos y programación SQL.

<https://www.cepotel.org.ar/wp-content/uploads/2024/02/Introduccion-a-Bases-de-Datos-y-Programacion-SQL.pdf>

Dalmasso, C. (2022). Introducción a las bases de datos. Escuela Superior de Comercio, Universidad Nacional de Rosario.

https://esupcom.unr.edu.ar/bv_tics/archivos/informacion_complementaria/ofimatica/base_datos.pdf

Marqués, M. (2011). Bases de datos. Departament d'Enginyeria i Ciència dels Computadors, Universitat Jaume I.

SIPECOM. (2026, 12 de enero). Bases de datos: El gran salto de los archivos físicos a la nube inteligente. <https://sipecom.com/2026/01/12/bases-de-datos-el-gran-salto-de-los-archivos-fisicos-a-la-nube-inteligente/>

Torregrosa García, B. (2020). Modelos de datos. Universitat Oberta de Catalunya.

<https://openaccess.uoc.edu/server/api/core/bitstreams/cd624356-d7bd-4022-888f-e2324ee440aa/content>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional 06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Olga Constanza Bermúdez Jaimes	Responsable de línea de producción Huila	Dirección General
Joinner Enrique Osorio Martínez	Experto temático	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Paola Alexandra Moya	Evaluadora instruccional	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Yerson Fabian Zarate Saavedra	Diseñador de contenidos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Cielo Damaris Angulo Rodríguez	Desarrollador full stack	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Alejandro Delgado Acosta	Intérprete lenguaje de señas	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Cristhian Giovanni Gordillo Segura	Intérprete Lenguaje de señas	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Carlos Eduardo Garavito Parada	Animador y productor multimedia	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Maria Carolina Tamayo Lopez	Locución	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
German Acosta Ramos	Locución	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Aixa Natalia Sendoya Fernández	Validador de recursos educativos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila

Nombre	Cargo	Centro de Formación y Regional
Ricardo Oliveros Zambrano	Validador de recursos educativos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Anyerson Wilfredo Pizo Ossa	Evaluador para contenidos inclusivos y accesibles	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Daniel Ricardo Mutis Gómez	Evaluador para contenidos inclusivos y accesibles	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila